Memento - Platform Specification

Version 1.0 - 2026-10-28 - Selected React frontend, Bun backend, SQLite/PostgreSQL adapters and Podman development profile.

Read Product, Experience, Domain and Technical for the behaviour and contracts realized here. This document selects implementation technologies and boundaries. Selection is not evidence that the application, dependency compatibility, performance or deployment gates have passed.

1. Adopted platform decisions

The user authorizes the frontend technology baseline in spikes/02-react-renderer-comparison/package.json, a backend running on Bun, SQLite for normal development/tests, PostgreSQL for deployment, and Podman development containers. These replace the earlier Rust/Axum/Tokio/SQLx platform and the previous mandatory Tailwind/shadcn/React Router/ELK selections.

The spike manifest and its lockfile are implementation references, not the product's semantic model or production package. Its renderer can inform the application, while its mock backend and experimental data representations require replacement by the specified domain/application use cases. The third collaboration spike and its OpenSpec/Yjs proposal retain experimental scope; they do not override the platform or select CRDT merging for Memento.

Concern	Selected baseline
Browser	React and React DOM with TypeScript, client-rendered.
Board	React Flow (`@xyflow/react`) with owned custom cards/controllers and SVG/DOM presentation.
Drawing	`perfect-freehand` for stroke outline generation, mapped to board-owned durable ink.
Icons/font	`lucide-react`; locally bundled `@fontsource/ibm-plex-sans`.
Styling	Repository-owned CSS/theme tokens/components, following the mockup and renderer spike. No additional CSS framework is required by the selected manifest.
Build/dependency tooling	Bun package manager/scripts and committed lockfile; Vite build/dev server with React plugin; explicit TypeScript checking.
Backend	TypeScript on Bun; `Bun.serve` HTTP and WebSocket adapter, application composition and durable workers. No Node or Rust server process is required.
SQLite implementation	Separate repository/unit-of-work adapter using `bun:sqlite`, normal test/development default.
PostgreSQL implementation	Separate repository/unit-of-work adapter using Bun SQL, mandatory deployment store and real parity-test target.
Tests/lint	Vitest and Playwright for frontend/component/browser coverage; Bun test for backend/domain and adapter suites; oxlint and TypeScript checking.
Development containers	Podman, Containerfiles and OCI images, retained named/bind volumes and loopback ports.

The backend's initial toolchain pin is Bun 1.4.0, the locally available version inspected on 2026-10-28. Validate application/driver compatibility at bootstrap; changing the pin is recorded with lockfile/build/parity evidence. This is not a measured deployment certification.

2. Frontend dependency baseline

The following are declared manifest versions/ranges, not claims about every transitive resolved package. Preserve the spike lockfile as the experiment's evidence; the application gets its own committed lockfile and license inventory. Reconcile compatible changes explicitly rather than silently selecting latest versions.

Runtime dependency	Declared version
`react`, `react-dom`	19.3.1
`@xyflow/react`	12.12.0
`lucide-react`	1.60.0
`perfect-freehand`	1.2.3
`@fontsource/ibm-plex-sans`	5.3.0

Development dependency	Declared version/range
`typescript`	~6.0.3
`vite`	^8.3.4
`@vitejs/plugin-react`	^6.1.2
`vitest`	5.0.5
`@playwright/test`	1.64.1
`oxlint`	^1.88.0
`@types/react`, `@types/react-dom`	^19.3.0
`@types/node`	^24.19.2; tooling declarations do not select a Node backend.

Bun-specific backend type declarations are selected/pinned compatibly during bootstrap. No database ORM, additional server framework or model SDK is selected solely by this manifest. The baseline uses native server/driver adapters behind context interfaces; introduce another dependency only for a defined responsibility and with notices/compatibility evidence. All mandatory application components remain open source; hosted proprietary collaboration is not a dependency. Model endpoint/weight licensing is a separate deployment decision.

3. DDD code organization and dependency rules

The following is the target layout, not a claim that directories already implement the product:

```
apps/
  web/src/                # shell, features, board adapter, API client
  server/src/             # Bun entry, HTTP/WS, composition, worker entry
packages/
  memento/src/
    workspace-access/
    collaborative-knowledge/
    visual-collaboration/
    agent-participation/
    connected-work/
  contracts/              # public request/result/event schemas and client types
  storage/src/
    sqlite/
```

```
    postgres/
migrations/
  sqlite/
  postgres/
deploy/
  app/                    # application Containerfile/run profile when implemented
  test-environment/       # existing independent service package
```

Domain code uses plain validated TypeScript values and deterministic operations; it does not import Bun server requests, React/React Flow node types, SQL rows or provider SDK messages. Application use cases establish authorization, transaction and orchestration boundaries. HTTP, storage, model, connector, password/session and clock/ID facilities are adapters. Use small explicit ports only where an external concern exists, without a generic repository/service for every value object.

Context exports expose the Domain Specification's command/query interfaces. Enforce dependencies through TypeScript project/package boundaries and import checks. A consumer cannot import another context's private persistence mapping. Cross-context transactions are coordinated at application composition through a shared unit of work and each owner's published interface. Shared types contain IDs/references/operation metadata, not one mutable universal entity.

The two database adapters share repository contracts and contract tests. The application does not branch its business rules on the selected engine. Internal table names may be context-prefixed, but transport and model tools use published DTOs. Runtime schema validation is required even when browser/server share TypeScript types.

4. Browser application and design system

Vite builds static assets; Bun serves those assets and APIs under one deployment origin, or an explicitly configured reverse proxy does. Development Vite proxies API/WebSocket traffic to Bun. The app owns route/URL parsing and navigation contracts for Access, Home, workspace/Board, Templates and global/board Activity. A specific routing library is not mandated by the referenced package; add one only through an explicit dependency decision.

Feature modules cover access/members, Home/return state, templates, composition/inspection, Sources/publication, Agents, Activity and Traceability. A shared UI layer owns controls and theme tokens. Ordinary CSS variables define surface/text/border/accent/paper palettes, spacing, typography, focus, depth and responsive behavior. Reuse Node shell, reference-information card, status badge, contextual action, modal, form error and toast patterns. Match the Experience Specification's information/action distinction and canonical panel/drawer/dialog behavior.

Import the selected font locally through the build; browser runtime must not rely on a remote font CDN. Lucide icons provide consistent control symbols with accessible names; decorative icons do not replace labels. `perfect-freehand` produces rendering geometry for recorded strokes; the persisted source points/style/identity are owned by the composition model, not an opaque library cache. Measure stroke/card density with actual intended rich content.

Keep authoritative server state, pending local interaction, personal durable markers and ephemeral view state separate. Favorites/visits/read markers persist per member through the server. Theme/view/selection/open-panel preferences can remain browser-local; they grant no authority. Content, Decisions, conversations, captured evidence and outcomes cannot rely on localStorage for durability. Optimistic state stays visibly pending until accepted; conflicts preserve the draft.

5. React Flow as a presentation adapter

Map node IDs to Placement/VisualElement representations with explicit domain references and revisions. A source projection includes the selected ObservationId; it cannot subscribe directly to a global latest source value. Removing a renderer node invokes RemovePlacement, not DeleteRecord. Note colour belongs to visual presentation and never increments content evidence revision.

Custom nodes support freeform inline Notes, captured sources/tasks, Proposals, Decisions, Actions, Text and Shape. Gestures inside editable text/buttons must not trigger canvas pan/drag. Frame membership is explicit and batches member arrangement through validated commands; geometric overlap alone does not assign permanent membership. `perfect-freehand` and SVG render ink/link paths in the same board coordinate system.

A handle drag creates a visual link only. Semantic supports/origin, Decision supersession, results-in and Action prerequisite use the contextual domain commands and are labelled/inspected as semantic records. Traceability is a composed information dialog/query, not a second React Flow document or required ELK layout. The platform does not require automatic layout or CRDT merely because the library can display graphs.

Internal collaboration follows Technical section 3: server-accepted HTTP operations, ordered committed WebSocket updates, explicit stale-write conflicts and recoverable snapshots. Yjs/Yrs are not selected for production. A later proposal must define exactly which fields merge and preserve authority, provenance and acknowledgement before changing this contract. Visual undo sends a new revision-aware operation; it cannot invoke knowledge/external reversal.

6. Dual storage and backend runtime

Choose storage explicitly at composition time. Public use cases are asynchronous; the SQLite unit-of-work implementation executes synchronous statement/transaction work without `await` inside `bun:sqlite` transaction bodies. The PostgreSQL implementation awaits queries through the same transaction connection. They implement Technical section 4's protected authority/graph reads, constraints, deduplication, publication and durable work rules.

Configuration	Required behavior
`MEMENTO_PROFILE=development`	Defaults to SQLite and a retained writable file; Bun native development or the Podman app image is supported.
`MEMENTO_PROFILE=test`	Isolated SQLite by default; shared adapter suites explicitly select PostgreSQL as an additional real-engine target.
`MEMENTO_PROFILE=deployment`	Requires PostgreSQL and valid deployment secrets/origin settings; never silently starts with SQLite.
`MEMENTO_STORAGE=sqlite`	Uses the resolved `MEMENTO_SQLITE_PATH`, including the development/test default; open flags/permissions and required pragmas are checked at readiness.
`MEMENTO_STORAGE=postgres`	Requires `MEMENTO_DATABASE_URL`; pool/timeout/SSL configuration is explicit and credentials are not logged.
Invalid profile/driver/settings	Startup/readiness fails with safe diagnostics rather than changing storage or creating an accidental empty database.

The intended default development path is `./var/memento.sqlite`, ignored by Git. Container profile mounts the containing directory at `/data`, covering database/WAL/SHM files; its path becomes `/data/memento.sqlite`. In-memory SQLite is useful for isolated tests, not restart or realistic concurrency evidence. Enable foreign keys per connection and use WAL/FULL durability for file profiles. PostgreSQL version/image is pinned when its deployment/parity profile is authored; the existing Nextcloud PostgreSQL 17 package is not automatically Memento's selected database instance.

Migrations have paired logical versions and engine-specific statements as required. Use transaction-scoped repositories, explicit conversion/validation and shared contract tests. SQL is parameterized; dynamic identifiers come from fixed validated mappings. Do not assume SQLite type affinity, PostgreSQL JSON operators, collation or transaction timing are interchangeable. Adapter-specific locking may differ, while tested business outcomes must agree.

Bun handles HTTP/WS and asynchronous inference/provider I/O. Avoid CPU-heavy synchronous hashing or long database scans on the main loop; apply bounded payloads and use asynchronous facilities/workers where measured need requires them. Bun native WebSocket publication is a transport optimization, not a durable outbox. Background work is database-backed, versioned and safely resumable.

7. Agent and connector integration

Model and provider adapters translate external APIs into Domain interfaces. Provider SDK types never become aggregate state. A runtime records actual supplied messages, ContextSelection and granted tools; every tool invokes authorized application use cases. Credentials remain server-held. Select the model provider/client during its implementation change and record actual inference evidence and operating configuration.

Concern	Selected service / scope
Local mail	Stalwart Community, accessed by the validated adapter protocol; Nextcloud Mail is the independent participant client.
Tasks	Nextcloud Deck, with explicit native assignee/due/completion mapping and actual create/reconcile evidence.
Independent chat/files	Nextcloud Talk/Files remain external environment clients, not general Memento connectors.
Source reads	Scoped on-demand browse/import/refresh/publication; no always-running record synchronization.
Writes	Exact reviewed create-task/send-reply Actions, with durable identity and uncertainty recovery.

Endpoint/account values in the mockups are illustrative. The Test Environment defines actual local ports/accounts, native service versions, startup/reset and independent client checks. Application connector configuration references those settings explicitly. Provider outage changes operation availability, not historical evidence. Recovery and Action reconciliation must not become an unrequested background refresh of unrelated source records.

8. Podman containers and operating profiles

Use a Containerfile to build an OCI application image with Podman; no Docker daemon is required in development. The target build command shape is `podman build --format oci -f deploy/app/Containerfile -t localhost/memento:<version> .`; the Containerfile and executable runbook are implementation deliverables, not files created by this specification rewrite.

The image contains a pinned Bun runtime, server application and built browser assets. Build with locked dependencies, type-check explicitly and retain required notices. Runtime uses an unprivileged user, explicit writable data path and health/readiness endpoints. Keep secrets in ignored/private configuration or runtime secret injection, never in the image/repository. Native packaging/compilation is optional only after measured compatibility evidence; the basic runtime deployment is sufficient.

Normal local application runs retain SQLite in a named/bind volume and publish only a documented loopback port. Real external services remain in their existing separate supervised container. Real PostgreSQL parity testing uses a separately named database/container and isolated data; it does not write Nextcloud's database. Deployment uses PostgreSQL with controlled migrations/backups; a deployment orchestrator is not selected by the Podman-development decision.

Podman start/stop/recreate checks cover permission to write the volume, SQLite sidecars/consistent restart, readiness, worker/outbox recovery and static/API same-origin behavior. Record macOS host/Podman VM architecture and image digest; successful ARM64 operation is not Intel/Linux acceptance. Existing service evidence remains dated and bounded; updating a documented media-format command does not retroactively validate a rebuilt image.

9. Verification, pinning and decisions

Run domain/use-case tests without HTTP, renderer, model or external provider dependencies. Run shared persistence suites against both real engines. Type-check both frontend and backend separately from Vite transpilation. Verify public schemas/client mapping, dependency boundaries and actual browser journeys; run real provider/inference/deployment profiles for their respective ACs. Test selection and evidence belong to the authorized slice, not a blanket claim from a build.

The initial product scripts should provide check/typecheck/lint/test/build/migrate/seed/start and both adapter-test targets when implementation introduces executable tooling. Their exact commands are recorded then; the specification does not claim scripts already exist. Frontend Vitest and backend Bun test have different responsibilities. Tests of Node-based experimental spikes remain separate from Bun application acceptance.

Remaining implementation choices are the exact locked application dependency graph, backend type declarations, migration runner, provider/model adapters, PostgreSQL deployment version, credential storage integration and measured host/browser/graph budgets. Their change designs must preserve R/S stories, DR rules and Technical AC gates. No open technology choice permits silently restoring Rust, continuous external sync, renderer-owned semantics or PostgreSQL-only development.

Primary implementation references

Consulted 2026-10-28; these establish upstream API capabilities, not Memento acceptance:

- Bun HTTP server and WebSockets for transport adapters.
- Bun SQLite for synchronous SQLite transaction boundaries and WAL handling.
- Bun SQL for PostgreSQL connection/transaction facilities.
- Bun password hashing for asynchronous credential handling.
- SQLite transactions and PostgreSQL isolation for engine-specific concurrency design.
- Podman build for Containerfile/OCI builds.